Technical introduction to PSOC™ Edge E84 features

Chapter 3: Sensor hub

About this document

Scope and purpose

This document is the training manual for the technical introduction to PSOC™ Edge E84 features. This manual helps in getting started with different applications leveraging key features for PSOC™ Edge, using ModusToolbox™ software.

The training is divided into multiple chapters covering topics like audio, graphics, machine learning, among others.

Chapter 3 discusses PSOC™ Edge's sensor hub features.

Intended audience

The document is intended for design engineers, technicians, and developers of electronic systems.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Prerequisites **4**
3 Code example: Sensor hub IMU **5**
3.1 Objective 5
3.2 Description 5
3.3 Flow chart 6
3.4 Hardware overview 6
3.5 Project creation 7
3.5.1 Output 10
3.6 Modify application – read temperature 10
3.7 Modify application – add sensor fusion 12
3.8 Modify application – enable Euler angles 14
3.9 Conclusion 16
4 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ **17**
5 Appendix B: KIT_PSE84_EVAL details **20**
Revision history **21**
Disclaimer **22**

1 Introduction

This manual provides detailed instructions to create, configure, build, and run several application code examples on the PSOC™ Edge E84 MCU.

2 Prerequisites

See the **Prerequisites** section in the **Chapter 1 training manual** for a complete list of prerequisites and development tools.

3 Code example: Sensor hub IMU

3.1 Objective

This code example explains how to collect raw data from onboard sensors on a PSOC™ Edge E84 evaluation kit (EVK). The program uses the Arm® Cortex®-M33 CPU (CM33) to gather data at a rate of 100 Hz and then transmit it via UART to an external device. With this example, you can quickly get started with sensor hub data acquisition and gain insight into how to collect and process data from various sensors in real-time.

The user may optionally enable sensor fusion functionality for fusing the 6-axis sensor data using BSXlite sensor fusion library to compute the Euler angles and Quaternion vectors. With this extended use case, users can gain insight into how to process and fuse data from accelerometer and gyroscope sensors in real-time.

3.2 Description

In this code example, resource initialization is performed by the CM33 non-secure (CM33_NS) code. It configures the system clocks, pins, clock-to-peripheral connections, and other platform resources. It then enables the Arm® Cortex®-M55 CPU (CM55) using the Cy_SysEnableCM55() function. The CM55 is then put into its CPU Deep Sleep power mode.

The **BMI270** motion sensor is a low-power device that provides 3-axis accelerometer and 3-axis gyroscope measurements. This sensor is located on the EVK baseboard. The PSOC™ Edge E84 MCU uses I2C to communicate with the sensor and then converts the data into metric units. After conversion, the data is sent to UART. To interface with the BMI270 motion sensor, the PSOC™ Edge E84 MCU uses the BMI270-Sensor-API from Bosch Sensortec.

In the CM33 non-secure application, the clocks and system resources are initialized by the BSP initialization function. The retarget-io middleware is configured to use the debug UART, and the user LED is initialized. The debug UART prints a “Sensor Hub Data Acquisition” message on a terminal emulator. The on-board KitProg3 acts as a USB-UART bridge to create a virtual COM port.

In the second part of this lab, we will integrate the BSXlite sensor fusion library and we will show how to fuse the data from the accelerometer and gyroscope to compute Euler angles and Quaternion vectors.

Quaternions vectors are a more robust way to represent orientation. They use a four-dimensional unit vector to represent rotations in 3D space, and they do not suffer from gimbal lock. They are also more computationally efficient than Euler angles because they do not require trigonometric functions like sine and cosine.

Euler angles represent the orientation of an object in 3D space as a combination of three rotations around the X, Y, and Z axes. While they are easy to understand and interpret, they suffer from a problem called "gimbal lock" where certain configurations can cause the calculation to break down.

3.3 Flow chart

This following flow chart explains the code flow of this code example:

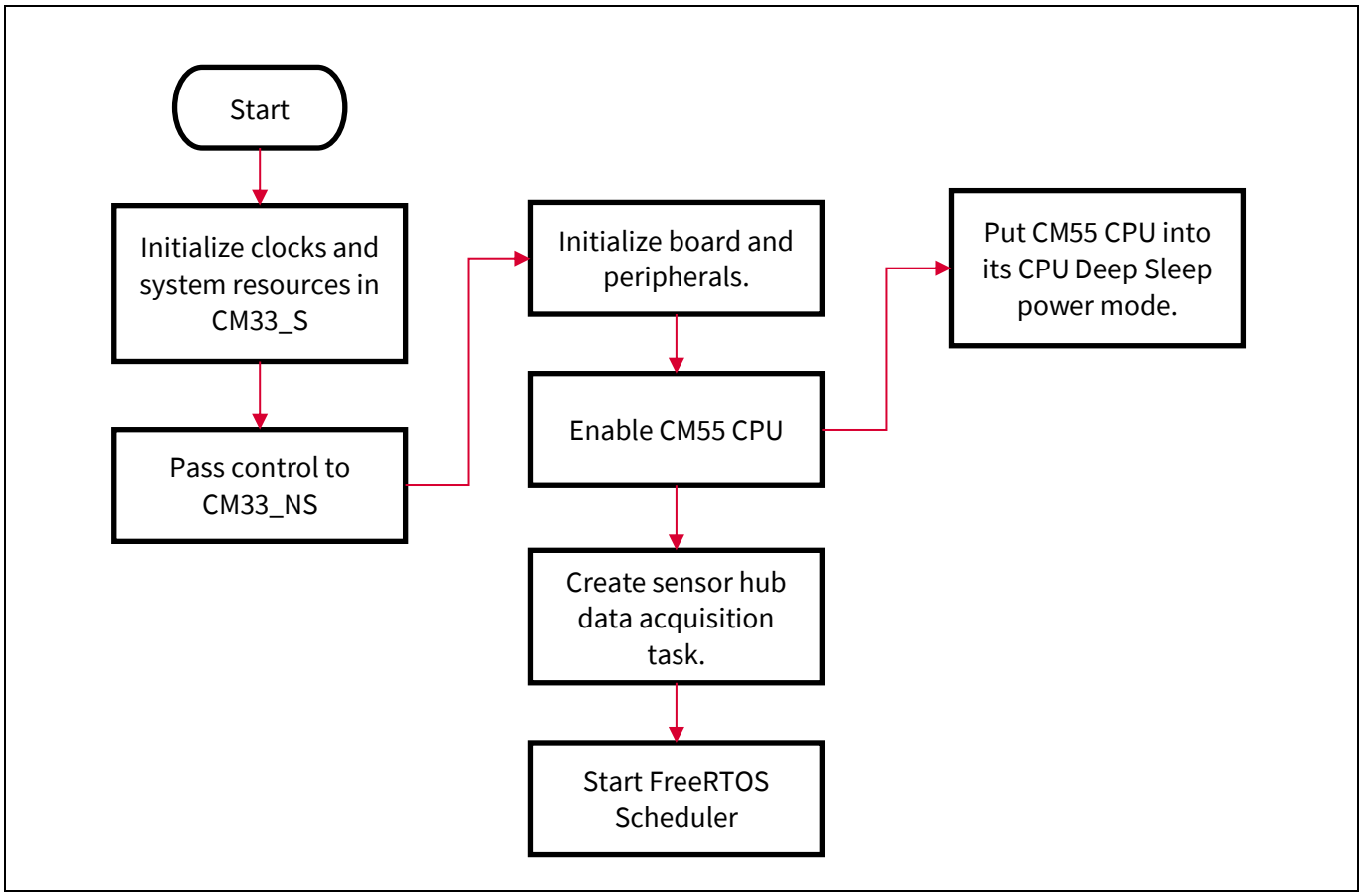


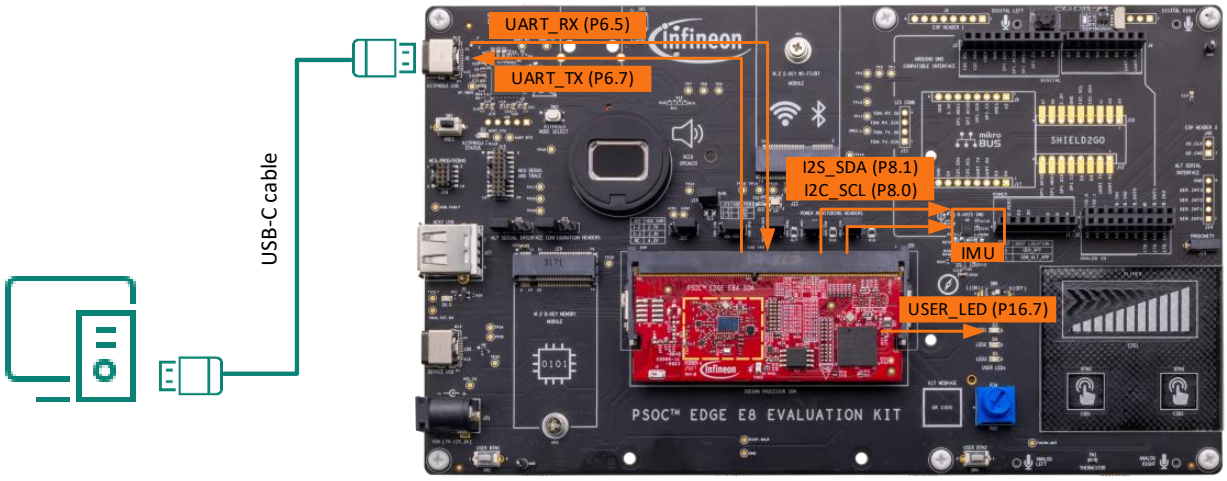
Figure 1 Sensor hub code example flow chart

3.4 Hardware overview

- UART pins:
  - P6_7 (UART_TX)
  - P6_5 (UART_RX)
- I2C
  - P8_1 (SDA)
  - P8_0 (SCL)

Note: This example requires BOOT SW in ON position.

See Appendix B: KIT_PSE84_EVAL details for more details.



Note: the signals shown are for illustration purposes, not exact routing

Figure 2 Hardware diagram for sensor hub data acquisition example

3.5 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge Sensor Hub IMU** application under the **Sensing** section.

   To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

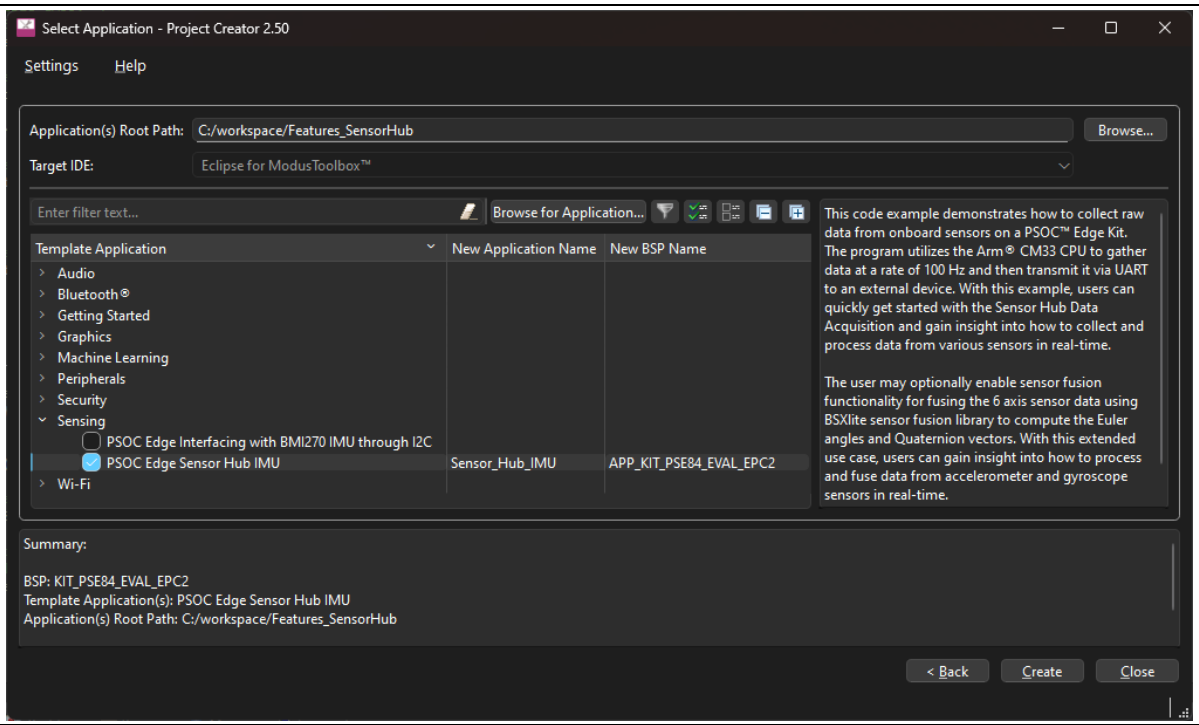


Figure 3 Create project for sensor hub IMU

3. **Note**: The following figure shows the key files and folders in this application.

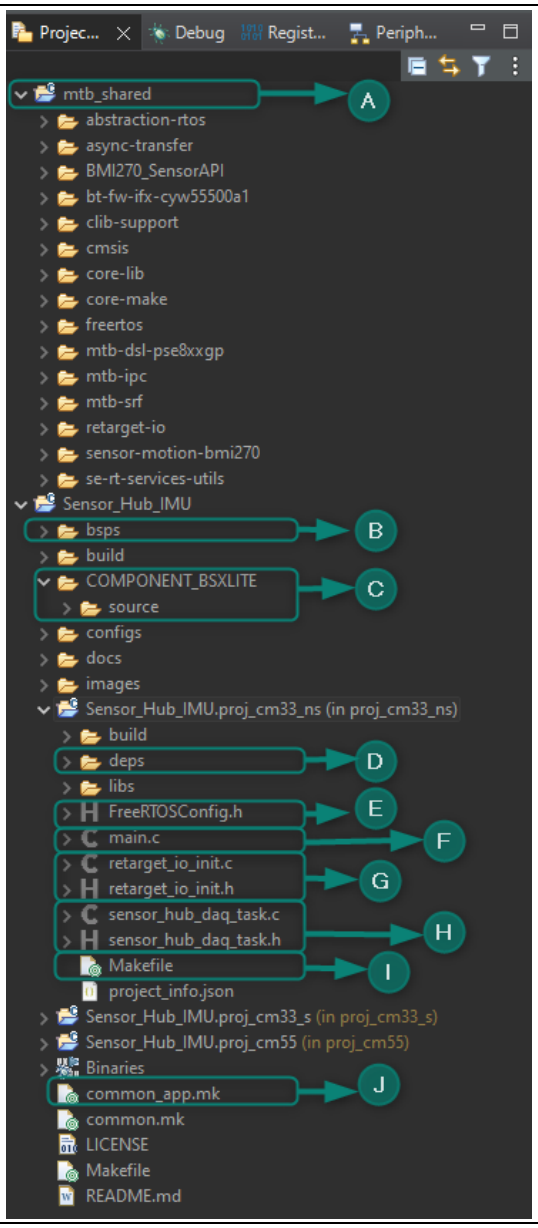


Figure 4 Sensor hub IMU project structure

a) mtb_shared

The mtb_shared directory is a folder that contains libraries that can be shared across projects in the workspace.

Observe that the project includes libraries to access the BMI270 IMU. These libraries can be included in other projects using the IMU.

b) bsps/TARGET_APP_KIT_PSE84_EVAL_EPC2 folder

The files provided by the board support package (BSP) are present here and are listed under the TARGET_APP_KIT_PSE84_EVAL_EPC2 sub-folders. All the input files for the device and peripheral configurators are in the “config” folder inside the TARGET_APP_KIT_PSE84_EVAL_EPC2 folder.

The “GeneratedSource” folder for the BSP contains the files that are generated by the configurators and are prefixed with cycfg_. These files contain the design configurations as defined by the BSP. From ModusToolbox™ 3.x or later, you can directly customize BSP configurator files for the application, rather than overriding the default files with custom files, since BSPs are completely owned by the application. The TARGET_APP_KIT_PSE84_EVAL_EPC2 folder also contains the linker scripts and the start-up code for the PSOC™ Edge E84 MCU used on the board.

c) components/COMPONENT_BSXLITE

This directory contains files to be used with the sensor fusion library, which will be added in the second part of this training.

d) deps

The deps folder is short for dependencies and contains the *.mtb* files for the application's direct dependencies. The *.mtb* files include a link to the library repository and the release tag that needs to be cloned for the application.

e) proj_cm33_ns/FreeRTOSConfig.h

The FreeRTOSConfig.h file is a configuration file that customizes the FreeRTOS kernel. It specifies application-specific configuration settings for the kernel and configures the FreeRTOS source code used in a project. A FreeRTOS application must have a FreeRTOSConfig.h header file in its pre-processor include path.

f) proj_cm33_ns/main.c

This file contains the logic for the main function. See Flow chart.

g) proj_cm33_ns/ retarget_io_init.c, .h

These files initialize the retarget-io middleware to use the debug UART port.

h) proj_cm33_ns/ sensor_hub_daq_task.c, .h

The .c file contains the function that initializes the I2C and BMI270 sensor, reads the sensor values, and prints them on the UART terminal.

The .h file contains the macros that configures sensor hub data acquisition task priority, stack size and delay for the sensor hub task.

i) proj_cm33_ns/Makefile

The makefile enables required components such as FREERTOS and RTOS_AWARE. It includes:

- Variables used to specify flags and pre-build and post-build commands.
- Path information for source code discovery, shared repo location, and path to the compiler
- The *common.mk* file for the application
- Path information to the *start.mk* file in the installation tools directory.

j) common.mk

This makefile is shared across all the projects. It enables a component to include configuration in all three projects.

4. Build and program the application.

3.5.1 Output

After programming, the application automatically starts.

A welcome message should be displayed, followed by accelerometer, gyroscope, and orientation data printed at a rate of 100 Hz.

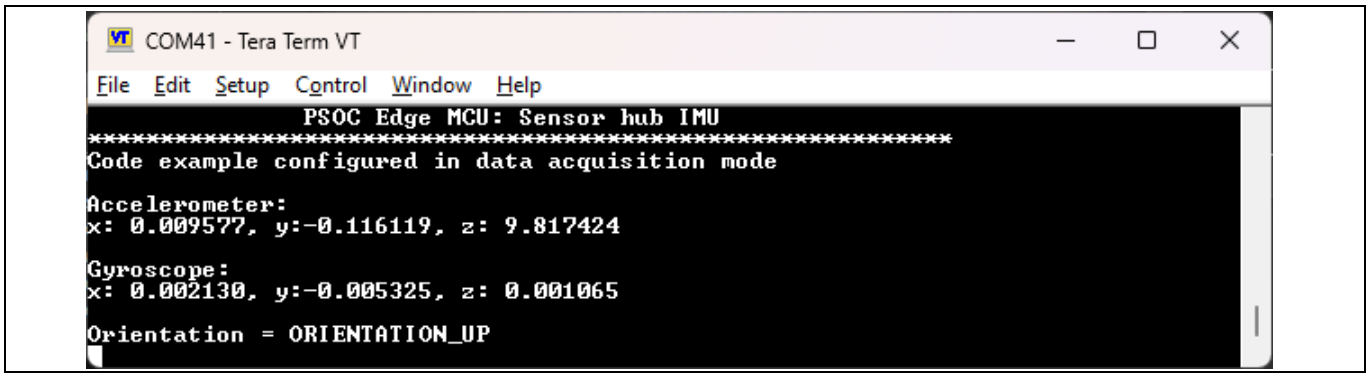


Figure 5 Sensor hub IMU output

Notice that the data from the accelerometer and gyroscope changes as you move the kit. The sensors measure motion and orientation, so any movement or change in the device's orientation affects their output.

3.6 Modify application – read temperature

As an exercise, you may extend this application and use the BMI270 sensor to read and display the temperature data in degrees celsius.

Hints for the advanced modification

- ModusToolbox™ includes a library supporting BMI270
- Use available APIs to gather and process data in the sensor hub task

Solution

Note: The solution is also available in the included source folder.

1. Declare the variables needed to store relevant data in *cm33_ns/sensor_hub_daq_task.c*:

Code listing 1 Declare variables

```
/*******************************************************************************
* Global Variables
*******************************************************************************/
static mtb_hal_i2c_t CYBSP_I2C_CONTROLLER_hal_obj;
static cy_stc_scb_i2c_context_t CYBSP_I2C_CONTROLLER_context;

static mtb_bmi270_data_t bmi270_data;
static mtb_bmi270_t bmi270;

static cy_en_scb_i2c_status_t initStatus;
```

Code example: Sensor hub IMU

```
static cy_rslt_t result;

/* To store the temperature data */
float temp_data_float;
uint16_t temp_data = 0;
```

Note: Observe the BMI270 API guide available here. The function mtb_bmi270_read_temp reads temperature data.

2. Add code to fetch and process the data from the temperature sensor in the sensor_hub_daq_task() function:

Code listing 2 Add code to gather and process temperature data

```
printf("Gyroscope: \n\rx:%9.6f, y:%9.6f, z:%9.6f \n\n",
         (double)x, (double)y, (double)z);

/* Advanced Modification to get temperature data Start  */

result = mtb_bmi270_read_temp(&bmi270, &temp_data );

if (CY_RSLT_SUCCESS != result)
{
    handle_app_error();
}

temp_data_float  = (float)((((float)((int16_t)temp_data)) /
    512.0) + 23.0);
printf("Temperature = %2.2f Degree Celsius \r\n\n", (double)temp_data_float);

/* Advanced Modification to get temperature data End  */
```

3. We have added two new lines to the debug console, so we must update the ANSI escape sequence. Modify the following code:

Code listing 3 Modify the escape sequence

```
/* \x1b[7A\x1b[42D - ANSI ESC sequence to get the cursor back
 * to the start. */
printf("\x1b[9A\x1b[42D");
```

4. Build, program, and run the application.
   You should observe the temperature information.

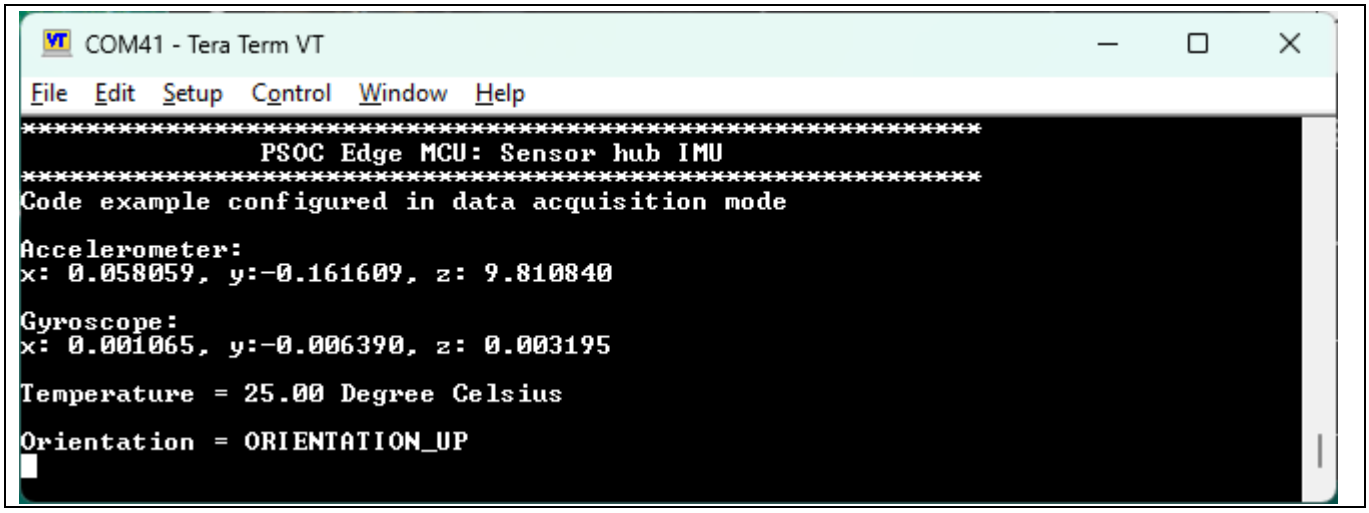


Figure 6 Output after adding temperature measurement

3.7 Modify application – add sensor fusion

IMU data is useful for a wide variety of applications, but it can become more powerful when fused to obtain additional information.

As a second exercise, you can add the Bosch BSXlite sensor fusion library to the application and use it to obtain quaternion vectors.

Hints for the advanced modification

- Download the Bosch BSXlite sensor fusion library and place it in the COMPONENT_BSXLITE folder.
- Enable sensor fusion in project configuration files.

Solution

Note: The solution is also available in the included source folder.

1. Download the Bosch BSXlite sensor fusion library.
   Enter the requested details on the website, read and agree to the license terms and proceed to download and unzip the library. Note that the BSXlite library for CM33 CPU is only available for GCC_ARM toolchain. The fusion functionality is therefore not supported by other toolchains.

2. Copy the following files from the unzipped folder to the *COMPONENT_BSXLITE* folder at the root of the project.
   - *./genericbsxlite_v1-0-2/[Generic]BSXlite_v1.0.2/lib/GCC_OUT/libalgobsxm33/libalgobsx.a*
   - *./genericbsxlite_v1-0-2/[Generic]BSXlite_v1.0.2/bsxlite_interface.h*

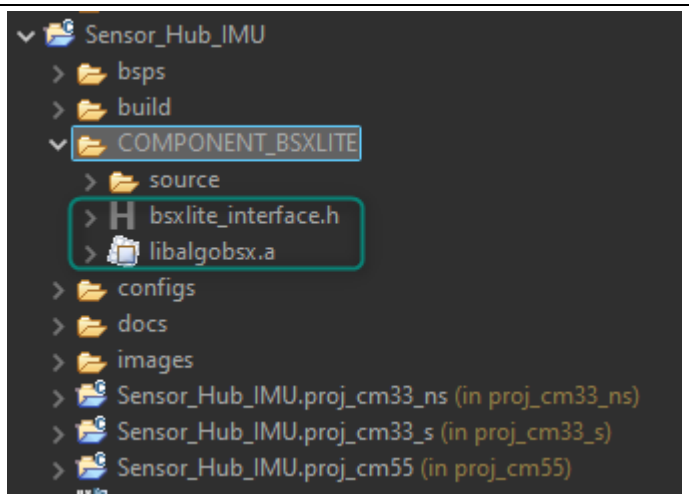


Figure 7 Project after adding sensor fusion library

3. Open *common.mk* and modify **CONFIG_APP_MODE** to **ENABLE_FUSION**.

Code listing 4 Declare variables

```
# Ex:
#     CONFIG_APP_MODE = ENABLE_ACQUISITION
#    or
#     CONFIG_APP_MODE = ENABLE_FUSION
#
CONFIG_APP_MODE = ENABLE_FUSION
```

4. Build and reprogram the device. You should observe the quaternion vectors in the terminal.

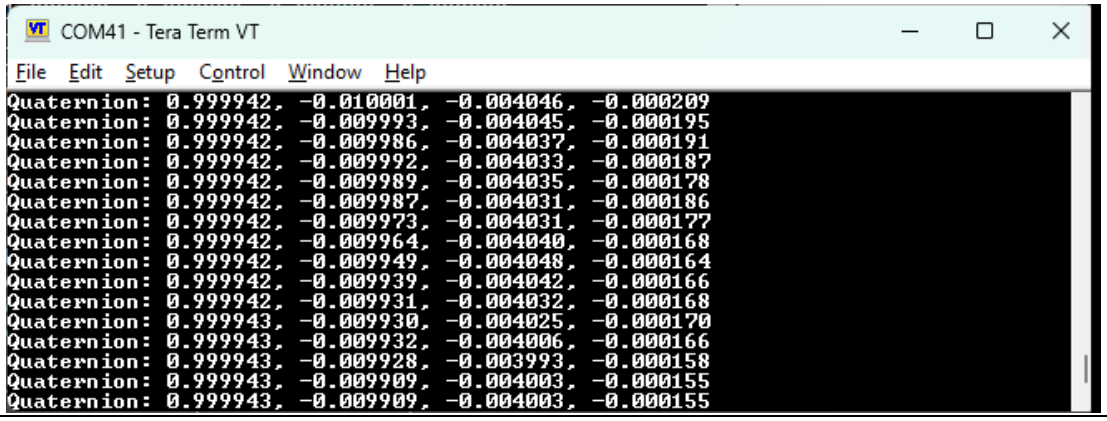


Figure 8 Quaternion vectors from sensor fusion

5. Disconnect the serial terminal and go to https://adafruit.github.io/Adafruit_WebSerial_3DModelViewer/. Select the corresponding port at 115,200bps and enable **Quaternions**.
6. Rotate the kit to observe the 3D image in motion, illustrating the sensor fusion use case.

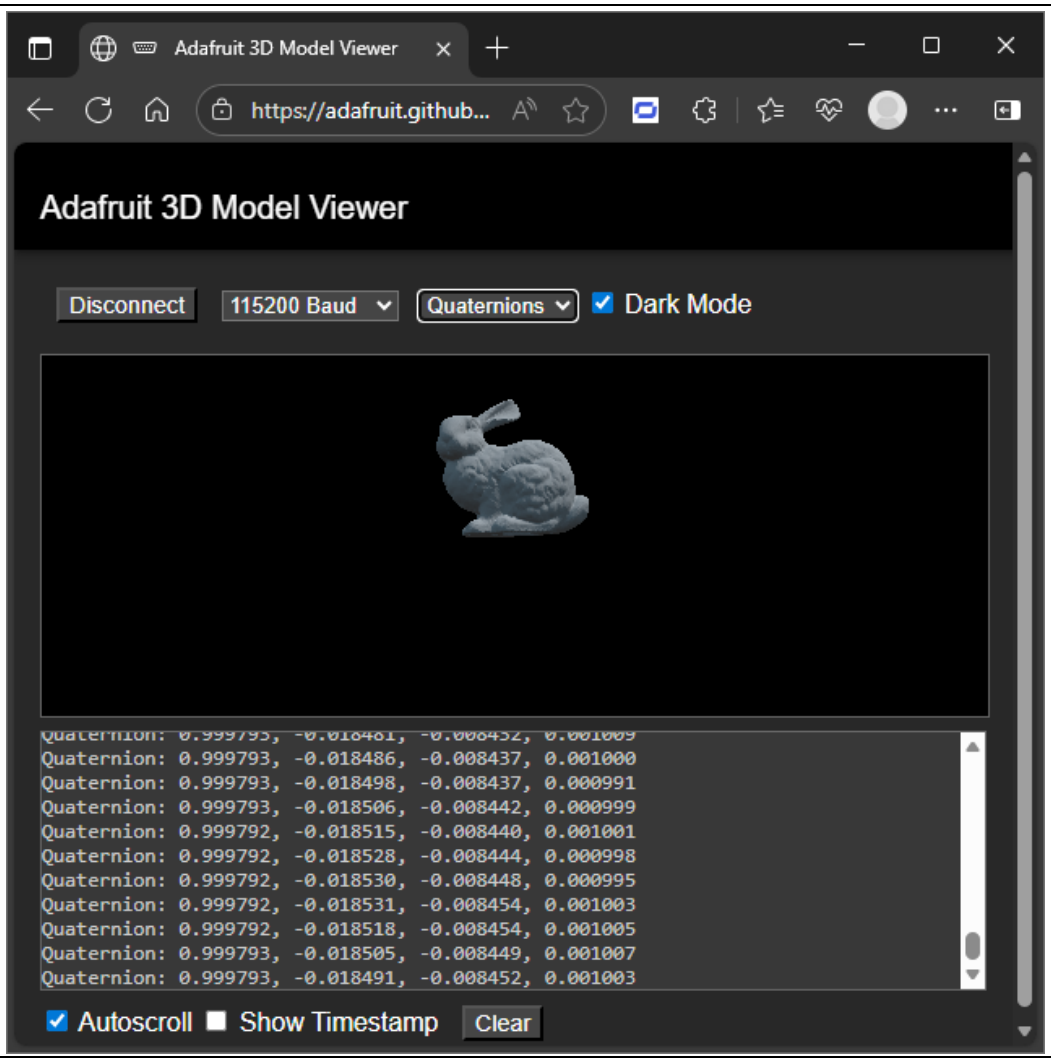


Figure 9 Adafruit 3D model viewer showing quaternion vectors

3.8 Modify application – enable Euler angles

We have shown how to use the sensor fusion library to calculate quaternion vectors which are a robust way to represent rotations in 3D space, but the library can also be used to calculate Euler angles.

As a third exercise, modify the sensor fusion application to obtain Euler angles instead of quaternion vectors.

Hints for the advanced modification

- The sensor fusion application is available in *sensor_hub_fusion.c/h*.
- The BSXLite library outputs Euler angles in radians per second; however, the 3D model viewer expects these values to be in degrees per second.

Solution

Note: The solution is also available in the included source folder.

1. Open *COMPONENT_BSXLITE/source/sensor_hub_fusion.c* and set **output_mode** as **OUTPUT_MODE_EULER**.

Code listing 5 Enable output mode as Euler angles

```
static const output_mode_t output_mode = OUTPUT_MODE_EULER;
```

2. The BSXLite library outputs Euler angles in radians per second; however, the 3D model viewer expects these values to be in degrees per second. To accommodate this, convert the Euler angles by multiplying them by a factor of 57.2958 (1 Rad × 180/π = 57.2958 Deg). To apply this conversion, make the following change in the same *sensor_hub_fusion.c* file.

Code listing 6 Calculate Euler angles in degrees per second

```
case OUTPUT_MODE_EULER:
    printf("Orientation: %f, %f, %f, %f\r\n",
        bsxlite_fusion_out.orientation.heading * 57.2958,
        bsxlite_fusion_out.orientation.pitch * 57.2958,
        bsxlite_fusion_out.orientation.roll * 57.2958,
        bsxlite_fusion_out.orientation.yaw * 57.2958);
    break;
```

3. Re-build, and program the updated application.
4. In the 3D model viewer, select the Euler angles option and rotate your kit to observe a phenomenon known as gimbal lock.

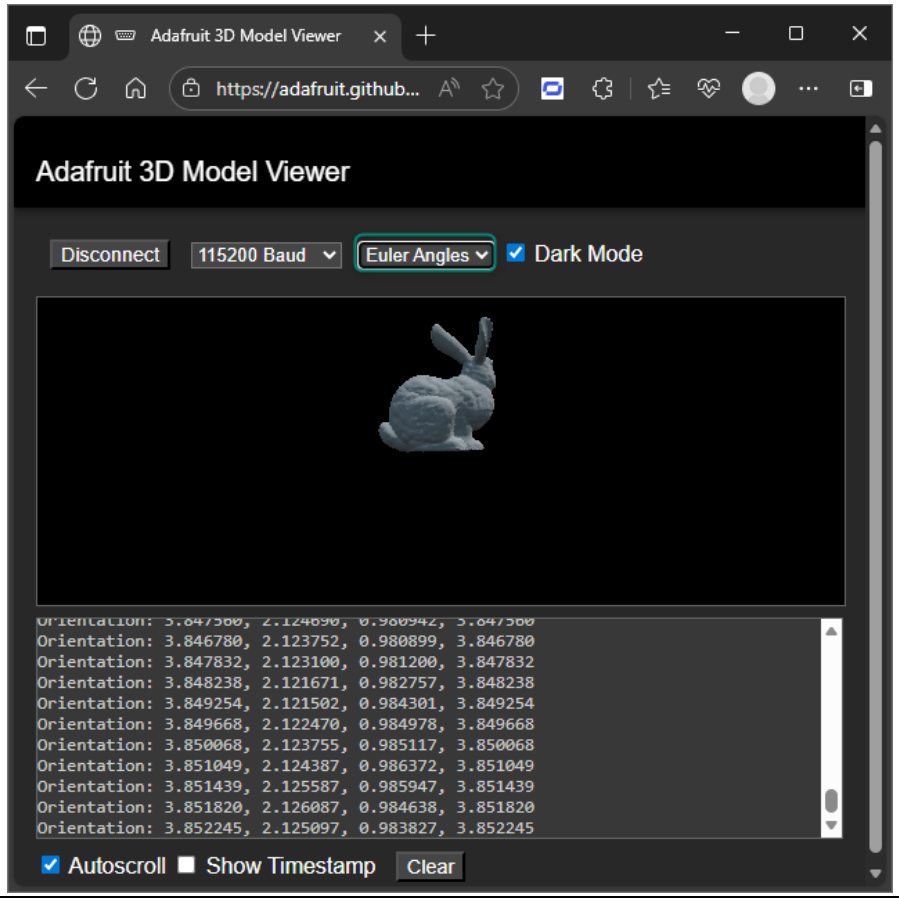


Figure 10 Observe Euler angles in 3D model viewer

5. **Note**: **Gimbal** lock occurs when the Euler angles approach a singular configuration, typically near 90 degrees, causing the representation of 3D rotations to become unstable and lose a degree of freedom. This results in an incorrect visualization of the sensor data. Read more about gimble lock here.

   Quaternion vectors do not suffer from gimbal lock, providing a more robust and accurate representation of 3D rotations. By using Quaternion vectors, you can avoid the issues associated with gimbal lock and ensure a more reliable visualization of your sensor data.

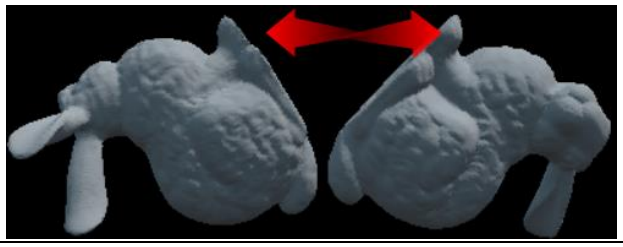


Figure 11 Gimbal lock

3.9 Conclusion

Through this exercise, we achieved three milestones. Firstly, we successfully created a PSOC™ Edge E84 MCU sensor hub application using ModusToolbox™, enabling us to acquire data from the sensor.

Secondly, we expanded the application's capabilities by fetching and printing the temperature data from the same sensor.

Thirdly, we expanded the capabilities of the application by adding support for the using BSXlite sensor fusion library to calculate quaternion vectors and Euler angles.

4 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

1. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**.

Note: If you have not installed ModusToolbox™ or Eclipse IDE, see the Prerequisites section.

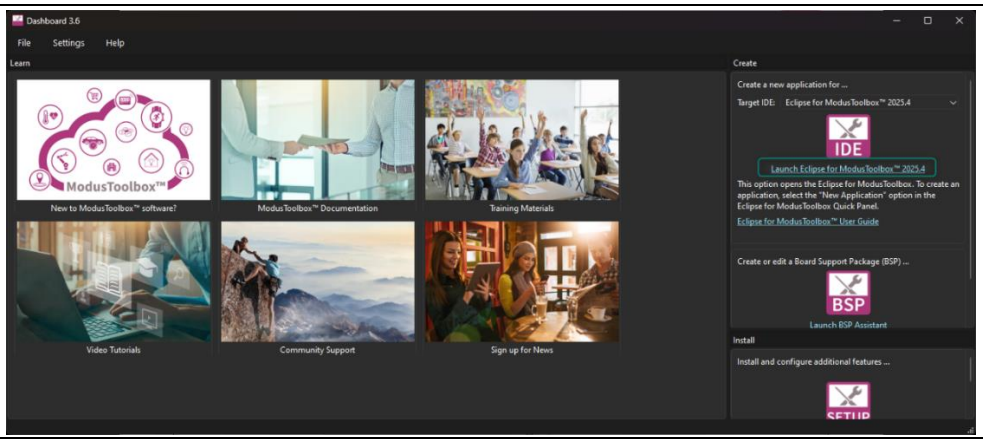


Figure 12 Open Eclipse IDE for ModusToolbox™ from Dashboard

2. Choose a **workspace directory** for your project.
   A folder will be created if the workspace does not exist.
   After selecting a suitable directory, click the **Launch** button.
   Note: It is recommended to keep the workspace path short. Windows has a 260 characters path-length limit and a long workspace path may cause build issues when creating some applications.

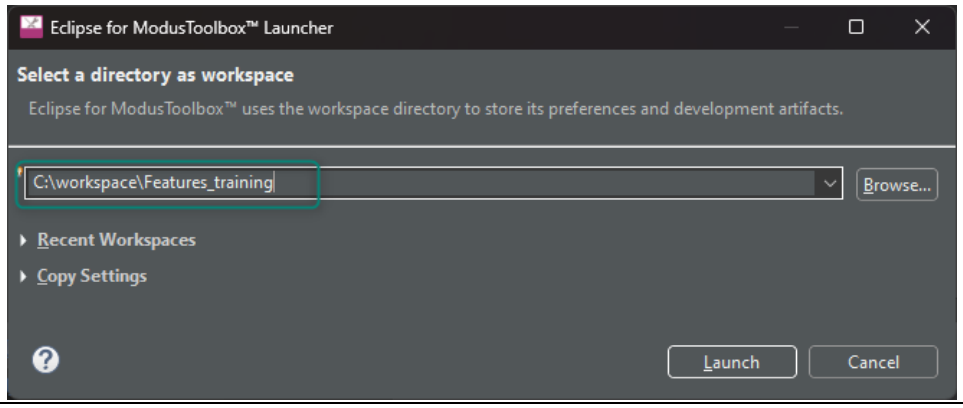


Figure 13 Choose workspace directory

3. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**.

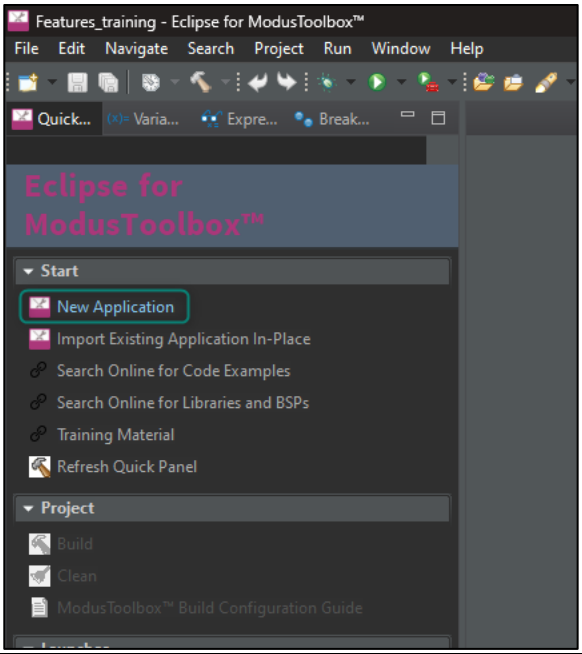


Figure 14 Create new application

4. Once **Projector Creator** launches, you will be prompted to select a board support package (BSP).
   Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next**.
   *Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*
   Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL which includes EPC2 devices by default.

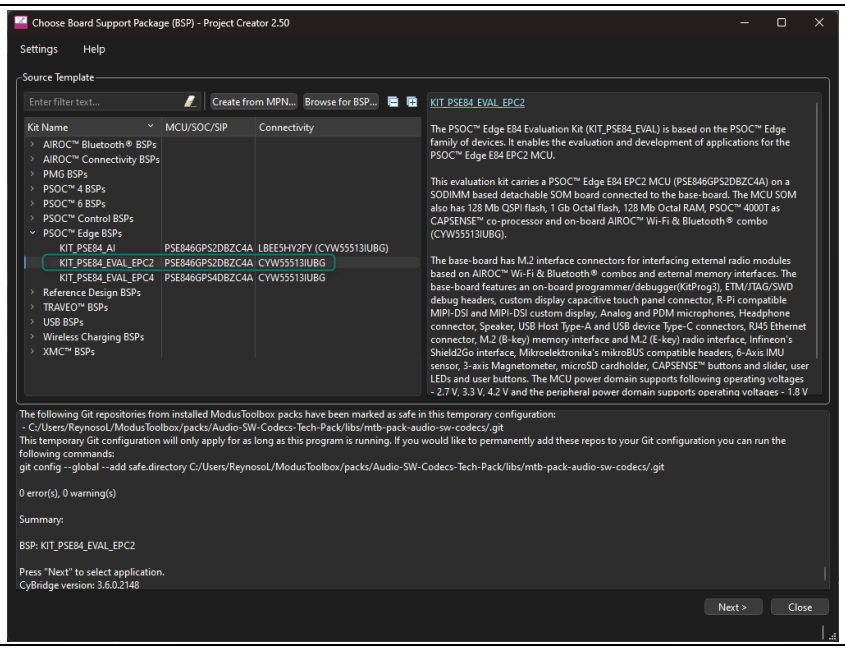


Figure 15 Selecting BSP in Project Creator

5. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples.
   Click the **checkbox** near the application, optionally rename the project, and then click **Create**.
   Figure 16 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**.
   Note: Windows has a 260 characters path-length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.

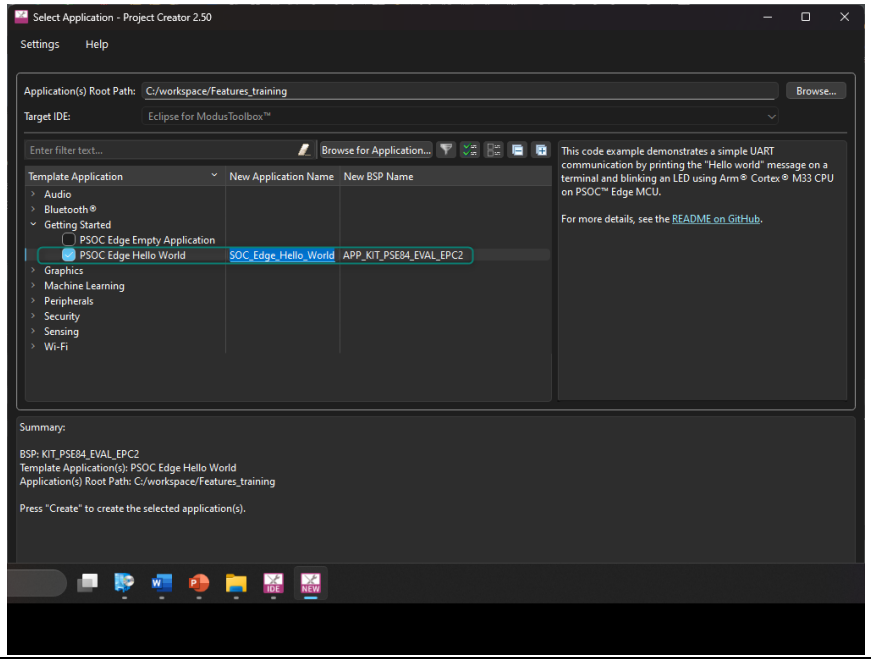


Figure 16 Selecting application

6. A new project is created, and with all the files visible in the **Project Explorer** window.

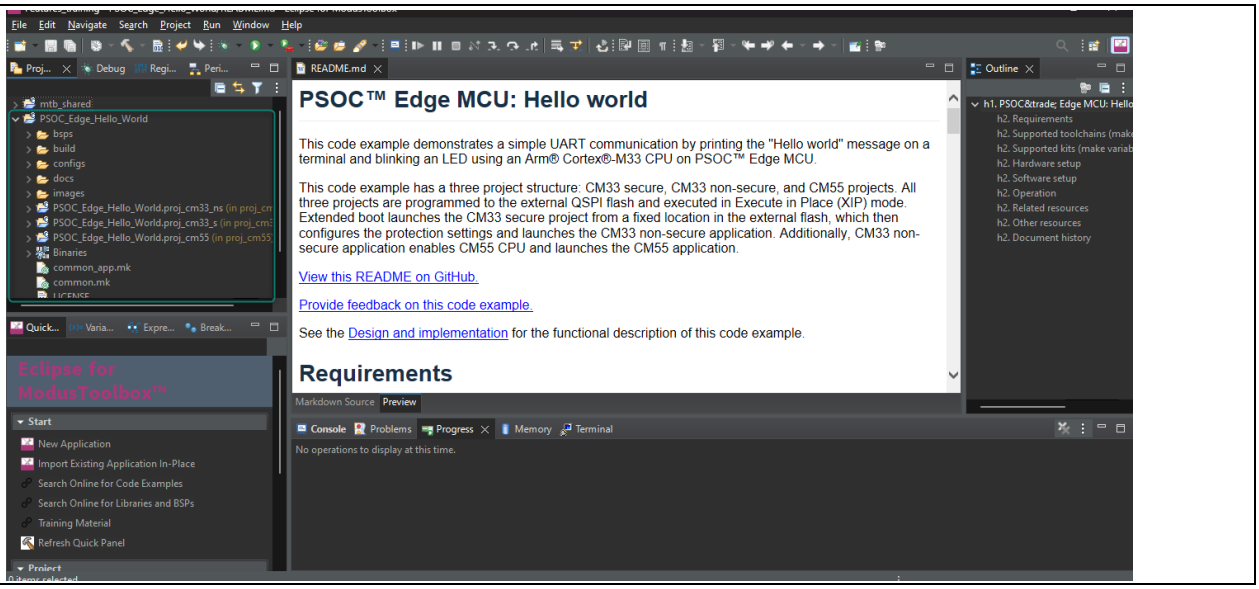


Figure 17 New application created

5 Appendix B: KIT_PSE84_EVAL details

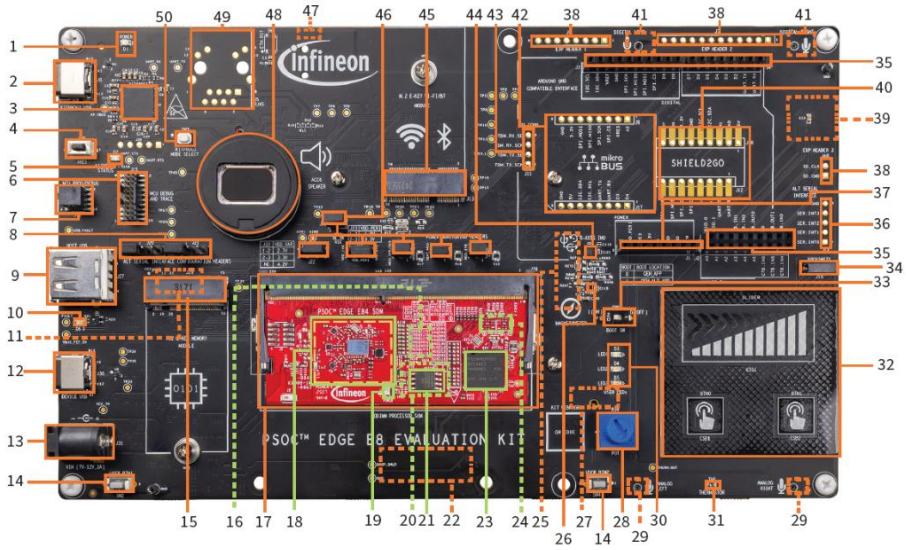


Figure 18 PSOC™ Edge evaluation kit (EVK) details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C Power Delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128-Mb Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor system-on-module (SoM) 260-pin SODIMM connector (J28)
18 AIROC™ CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor system-on-module (SoM) power LED (D3)
20 1-Gb Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128-Mb Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi-compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

* Footprint only, not populated on the board
** Component at the bottom side of the Baseboard
*** Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-01-15	Initial release.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-01-15

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com